

Choose Your Own AI Adventure

A guide to the LLM landscape for networking.

Eric Chou · Network Automation Advocate · Network to Code

AUTOCON S

THE NETWORK AUTOMATION CONFERENCE

\$ whoami

SPEAKER

Eric Chou Network Automation Advocate ·
Network to Code

Network engineer turned automation
person, now AI Enthusiast

Author, blogger, Podcaster

This talk is **not** a sales pitch.



\$ agenda.md

PART 01

Overview

Hype vs. reality.
Why "just let the AI
do it" doesn't work.

Set expectations

PART 02

The Landscape

Hosted, local,
hybrid.

what's out there

PART 03

The Path

Four steps. Trust →
Engineering →
Integration →
Evolution.

thing you can do now

PART 04

Closing

Framework. Two
rules. Q&A.

Next steps

The hype vs. the reality.

Network engineers don't want magic. They want the phone to not go off.

```
$ diff hype.md reality.md
```

– HYPE

"Just 'fix my BGP' "

+ REALITY

The phone rings at 2 a.m., do
you trust it and 'why not'?

AI is useful, but the demo and the day-job
are not the same thing.


```
$ echo $RISK_TOLERANCE
```

Network engineers are **extremely** risk-averse. Many of us would rather take the **2 a.m. maintenance window** to do a production change.

“The opposite of fragile is not resilience, it is antifragile.”

– Dr. Nassim Taleb

A FAMILIAR PATTERN

WALK

RUN

FLY

We did this with scripts. We did this with Ansible and Python.

AI is no different. It's a tool. Treat it like one.

Starting Point on the **map**.

Hosted, Local, Hybrid

\$ Llm-landscape

▣ 01

Hosted

Anthropic Claude

OpenAI GPT

Gemini

You rent. State of the art. Your prompts traverse someone else's data center.

▣ 02

Local

Ollama

Llama

Qwen

Mistral

You own it. Data never leaves the building. You also now run an inference stack – congrats, you're an AI ops team.

▣ 03

Hybrid

Routed by task

AWS Bedrock

Azure AI Foundry (Studio)

Sensitive ↔ local. Generic ↔ hosted. One API surface, multiple backends.

\$./hosted --rent

WHAT IT IS

The big public APIs. **Claude**. **GPT**. **Gemini**. You don't run them; you call them.

The strongest models, by a meaningful margin.

Zero infra. An API key and a OpEx budget.

State of the art moves about every six weeks. You get the upgrade for free.

THE CATCH

Your **prompts and configs** go through someone else's data center.

Your **security team** will have a strong opinion. They are correct to.

Cost is per-token. Easy to start, easy to surprise yourself.

You don't control the model.

Best for: prototyping, generic tasks, anything where the data isn't sensitive.

\$./local --own-the-box

WHAT IT IS

Ollama plus open weights. Llama, Qwen, Mistral, etc. The model runs on a box you control.

Data never leaves. Regulatory headaches go from existential to manageable.

Predictable cost. You bought the GPU once. The token meter doesn't tick. CapEx.

Open weights the 'mathematical weights' are public, but source code and trained data are still private.

THE CATCH

You now **run the stack**. Quantization, batching, scaling, on-call.

You will be a model version behind – sometimes a few.

Capability gap on hardest tasks: **real, narrowing, not zero.**

Hardware CapEx isn't free either – that GPU has a depreciation schedule.

Best for: regulated data, deterministic cost, anything that can't leave the building.

\$./hybrid --route-by task

WHAT IT IS

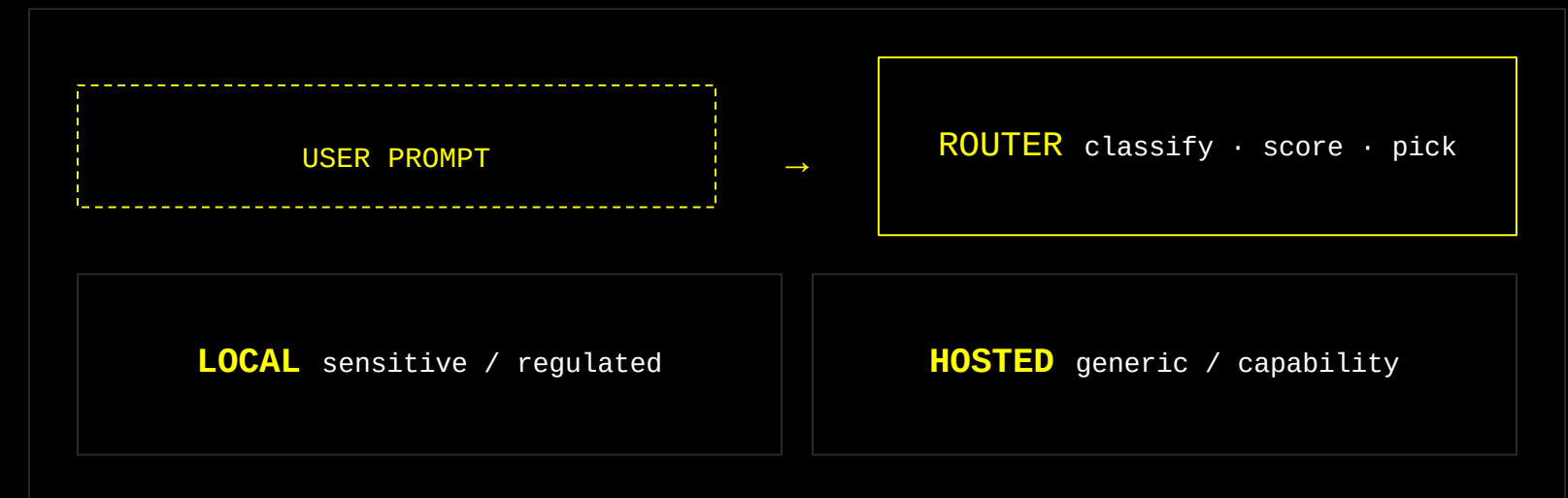
Routed by **task** or **sensitivity**. Often through **AWS Bedrock**, **Azure AI Foundry**, or a thin router you write yourself.

Sensitive customer configs → local model, in-VPC.

Boilerplate, scaffolding, summarization → hosted model, best in class.

One API surface. Your application code doesn't know which backend it hit.

A PICTURE



More Difficult to get started.

\$./pick --with-honestly

01

Data sovereignty & regulation

Can your configs and customer data leave the building? Sometimes the answer is "no" and that's the whole decision.

> Where does this data have to live?

02

Cost OpEx vs. CapEx

API tokens scale linearly with use. Hardware is a one-time bite with a five-year tail. They look different on different spreadsheets.

> Whose budget is this?

03

Skills & comfort level

Hosted is a billing problem. Local is an operations problem. Hybrid is both.

> Who's on-call for this?

04

Availability & blast radius

Public APIs go down. So do internal ones.

> What breaks when the model is gone?

>> §3 / NEXT STEPS RECOMMENDATIONS

Next steps. In order.

Trust, Engineering, Integration, then Evolution.


```
$ cat ./path.map
```

STEP 01

Trust & Guardrails

– pick your model, write your rails

Decide hosted / local.

Establish prompt hygiene.

Build the logs and traces.

STEP 02

Engineering

– beyond the magic trick

Zero-shot vs. few-shot vs.

RAG. Deterministic, schema-validated output. Confidence scoring.

STEP 03

Integration

– co-pilot, not autopilot

The Ideation → Review → Validation → Approval → Execution pattern.

STEP 04

Evolution & Feedback Loop

– make it learn

Human expert review of misses. System validation of outputs. AI evaluating other AI as a second-opinion layer.

\$ STEP 01 – Trust & Guardrails

DECISION

Hosted or local

Pick. Or accept that your security team has already picked for you, this determines your next step.

PRACTICE

Prompt hygiene & Guardrails

What you put in. How it's structured. What never goes in. System prompt or other restriction files.

INSTRUMENTATION

Visibility & traces

Log calls. Capture prompt, model, version, response, latency, cost.

\$ STEP 02 – Engineering

Example: perfect your data and prompt (input).

A

Zero-shot

> TRIVIAL · STATELESS · GENERIC

```
prompt: "Rewrite this
interface description in
plain English."
```

B

Few-shot

> YOU HAVE **5 EXAMPLES** OF RIGHT

```
examples:  - input: ...
            output: ...
            - input: ...
            output: ...
now do: ...
```

C

RAG

> ANSWER LIVES IN YOUR **DOCS** / CMDB

```
retrieve: top-k chunks stuff: into
context ask: grounded question cite:
sources back
```


\$ STEP 02b – More Engineering

Data Input and Output

Focus on data and results (output)

Structured output mode – ingestible data output

Sanity checks on values – IP in subnet? Interface exists?
VLAN in range?

Schema-validate Reject malformed responses; retry with a stricter prompt.

Confidence scoring – evaluation. Below threshold $\Rightarrow$ human.

\$ STEP 03 – Integration with Tools

GLUE

LangChain ...or plain Python.

Chain calls. Manage memory. Tool-use.
Whatever framework lets you describe a
pipeline as a pipeline instead of
nested if-statements.

FRONTEND

Streamlit ...or Gradio, or a quick web
UI.

Humans need somewhere to see what
the thing is doing. The goal is not
beautiful – the goal is **legible** . A
textarea, a diff, a button.

GOAL

A Network Co-Pilot

A human sits next to it.

\$ STEP 04b – Integration with Process

Don't trust. Always validate.

> AI

Ideation

Model proposes. Schema-validated JSON, candidate intents, candidate config.

> HUMAN

Review

An engineer reads it. Eyes on the diff. First sanity check.

> SYSTEM

Validation

Linters. Schema check. Dry-run. Diff against intent. Policy engine.

> HUMAN

Approval

A named person clicks the button. Their name on the change ticket.

> SYSTEM

Execution

And only now does anything touch a device. Push. Verify. Roll back if drift.

▣ Two human gates. Both required. Both named.

▲ Execution is the last 10% of the pipeline and 90% of the risk.

\$ STEP 04 – evolution & feedback

...you layer on the **feedback loop**

Human expert reviews the misses.

Code & system validation test cases,
digital twin, etc..

AI Evaluation optimized for speed, cost,
or accuracy.

3-Step Decision Framework. **And 2 Rules.**

A 3-step decision framework, and the two rule to live by.

\$./decide --in-three-questions

Q1

Where does the data have to live?

Picks **hosted**, **local**, or **hybrid** . If your security team has an opinion, this is already answered. Be honest about regulation and sovereignty before you negotiate with yourself about cost.

Q2

How complex is the task? Is it a complicated or complex problem?

Picks **zero-shot**, **few-shot**, or **RAG** . Default to the simplest one that works.

Trust, Engineering, Integration, Evolution

Q3

What's the blast radius if it's wrong?

Picks how many **humans** and how much **system validation and evaluation**.

```
$ cat ~/.rules
```

01

No AI-generated anything
touches production without a
human approval.

02

Start small. Iterate.

**“The opposite of fragile is not resilience,
it is antifragile.”**

– Dr. Nassim Taleb

Questions?

Eric Chou – Network Automation Advocate, Network to Code

› **LinkedIn**

<https://www.linkedin.com/in/choueric/>